Errors

שורה 43: ערך התכונה "מ" חייב להיות tools.md, שגיאה-1 – שונות/תרגומי-טקסטים-קדומים מוקף במרכאות - "..." או '...' או ’...’ או ”...”

Tools

> Part of the guidelines for translating ancient texts (translation-guidelines.md), split out of it on 2026-10-21 to keep the core file lean. Section number §10, as in the guidelines.

10. Tools (`tools/`)

Script	What	Usage
`ebl_fetch.py`	Downloads an eBL corpus chapter and builds `reading-edition.md`, `glossary.md`, `manuscripts.md`, `bibliography.md`, `chapter.atf`. Caches in `_raw/`.	`python3 -I tools/ebl_fetch.py --list` · `python3 -I tools/ebl_fetch.py L 1 4 "Standard Babylonian" XI sources/eBL/Gilgamesh-SB-XI`
`etcsl2md.py`	ETCSL TEI XML → Markdown (Unicode transliteration, subscript indices).	`python3 -I tools/etcsl2md.py <c-or-t.xml> <out.md> [url]`
`cdl2md.py`	ORACC CDL JSON (from a project zip) → Markdown with lemmatization.	`python3 -I tools/cdl2md.py <zip> <member> <out.md> <title> <url>`
`dfhg2md.py`	DFHG API JSON (Greek fragments) → Markdown.	`python3 -I tools/dfhg2md.py <in.json> <out.md> <title> <api-url> [frags]`

More tools (2026-10-16):

- **`check_bible_quotes.py`**: checks every backticked, vocalized Bible quotation that is followed by a reference against `תנך-מנוקד/`. It normalizes Unicode, allows prefixes and quotations that run on into the next verses, and defaults to Genesis. `python3 -I tools/check_bible_quotes.py <files...>`; `--show בראשית "ח כא"` prints a verse. **Run it before finishing any project.** Note: the Psalms and Proverbs files use a hyphen-minus for maqaf, so keep the `-` in quotations from them.
- **`ebl_fetch.py`** (fixed 2026-10-16): notes are now taken from the JSON (the ATF omits many), and range lines (`66-67`) are labeled.

More tools (2026-10-16, creation corpus):

- **`ebl_fetch.py`** (fixed again): a line with several **variant reconstructions** now prints all of them (**`N (variant, other manuscripts):`**) with their manuscripts; before, the second variant and its manuscripts were silently dropped. Re-run it on older eBL folders if a line looks incomplete.
- **Project-level verification pattern** (`סיפורי-הבריאה-המסופוטמיים/1-אנומה-אליש/verify.py`): checks the annotated entries against eBL's line list, the translation lines against the annotated `>` lines, every pair of eBL "parallel lines" (bracket-only differences listed separately), and a fixed passage against its table in `renderings.md`. Copy and adapt it for any eBL-based multi-tablet project. The same folder has `build_translation.py` (one reading file from per-tablet files) and `build_parallels_index.py` (merges per-tablet parallels tables into the corpus `parallels-index.md`, ordered by verse).

More tools (2026-10-16, creation corpus project 2):

- **`seal2md.py`**: a saved SEAL text page → Markdown (catalogue fields; the text table line by line, with bilingual continuation lines marked `↳`; SEAL's per-word lemmas; the translation). `python3 -I tools/seal2md.py <node-N.html> <out.md> <url>`. SEAL returns an error page to curl's default User-Agent: download with a browser UA (see the docstring).
- **`cdl2md.py --morph`**: the ORACC converter now prints each word's ePSD2 `morph` and base (`{ba.n.da:~;a,bi.a;nud}`), the Sumerian "check the stem" (§4.6).
- **`check_bible_quotes.py`** (fixed 2026-10-17): the loose book key dropped every ' and ו, so `יהושע` resolved to `הושע`; it now keeps the first letter and tries the exact name first.
- **eBL fragmentarium transliteration search**: `GET /api/fragments/query?transliteration=<signs>` finds every fragment with a sign sequence (e.g. `ul-le-gar-ra` → VAT 9307 = KAR 4). Use it to look for duplicates of a text before declaring it single-manuscript.
- **Project-level `verify.py` for a multi-text project** (`סיפורי-הבריאה-המסופוטמיים/2-טקסטי-בריאה-ועדן/verify.py`): per text, checks the annotated entries against the edition's line list (eBL reading edition or ETCSL transliteration) and the translation lines against the annotated `>` lines; `build_translation.py` there joins per-text files.

More tools (2026-10-17, law and treaty corpus):

- **`cdl2md.py --norm`**: adds a `norm:` line (the project's normalization of each word; SAAo, RINAP etc.) and prints the contextual sense after the guide word (`[(treaty-)oath=treaty]`).
- **`oracc_html2tr.py`**: a saved ORACC per-text HTML view → Markdown translation (units labelled with the transliteration lines they span, section headings as `##`). `python3 -I tools/oracc_html2tr.py <in.html> <out.md> <title> <url>`. Use it for any ORACC project: the zips have no translations.
- **Project tools in `חוקים-ובריתות/1-אמנת-הירושה-של-אסרחדון/`**: `build_data.py` + `make_parts.py` (align a composite with a later copy by the SAAo labels and cut per-part data files for the translators), `verify.py` (entries, one `>` per entry, **reserved words**, fixed formulas and key-lemma roots, identical normalizations), `build_translation.py` (clean translation with § headings and footnotes from a `הערת שוליים` field), `build_parallels_index.py` (corpus index, Deuteronomy grouped by chapter).

- **sources_sync.py** (2026-10-21): re-obtains the git-ignored sources. `status` / `fetch` / `verify` (work tier by default; `--all` adds the archive tier of old scans and photos; `--only REGEX`, `--dest DIR`, `--jobs N`); `snapshot` rewrites `sources/MANIFEST.json` where the files are. The recipes are in `sources/fetch-recipes.tsv`. It reports `CHANGED` when an online edition changed since our copy, and names the hosts blocked by a cloud allowlist. Text files are compared with their "generated/retrieved" dates blanked. Never extracts `.exe`/`.dll` from a zip. On a certificate error it retries with the public intermediates in `tools/certs/` (ORACC omits its intermediate; 2026-10-21), then falls back to curl with the same certificates.

When you write a tool that another session could reuse, put it here with a usage docstring and add a row to this table.

More tools (2026-10-17, the Laws of Hammurabi):

- **cdli_composite2md.py**: a CDLI composite ATF with `#tr.ts` / `#tr.en` lines → Markdown, optionally aligned line by line with one witness ATF (stele/tablet) by the transliteration sequence, and a JSON-lines file. Get the ATF from `https://cdli.earth/artifacts/<id>/json` (`[0]['inscription']['atf']`; witnesses' ATF inside `[0]['witnesses']`). `python3 -I tools/cdli_composite2md.py <composite.atf> <out.md> <title> <url> [--witness <w.atf> <siglum>] [--jsonl <out>]`.
- **Project tools in `חוקים-ובריתות/2-חוקי-חמורבי/`**: `lh_config.py` (files, topics, composite parser, stele alignment, SEAL mapping), `build_data.py` (per-file translator data: composite + stele + SEAL lemmas + rank-3), `verify.py` (entries per file with placeholders, § sections against the composite, reserved words, formula patterns, identical normalizations, translation = `>` lines; `--strict`), `build_translation.py` (topic and § headings, footnotes, placeholders). **Corpus builder** `חוקים-ובריתות/build_parallels_index.py` merges every project's Part tables (Exodus and Deuteronomy by chapter); it replaces the project-1 builder.
- **check_bible_quotes.py** (fixed 2026-10-17): references to `דברי הימים א/ב` were skipped; now matched.

שורה 43: ערך התכונה "מ" חייב להיות שונות/תרגומי-טקסטים-קדומים/שגיאה-1 – tools.md, מוקף במרכאות - "..." או '...' או ’...’ או ”...”

More tools (2026-10-18, the other law collections, in `חוקים-ובריתות/`): `laws_config.py` (projects LE, MAL, MALB, LI, UN: CDLI composite parser with exemplar links, ORACC MAL parser with the § units of the translation, files, TOPICS; Hebrew helpers), `laws_build_data.py KEY <dir>` (translator data per file), `laws_verify.py KEY [n...] [--strict]` (entries, § ranges, reserved and lemma-gated words, LH's fixed formulas for Akkadian and their equivalents for Sumerian, identical normalizations inside and across collections, translation = `>` lines), `laws_build_translation.py KEY` (topic headings by § or by composite label `@label`), `laws_lookup.py '<regex>' [--heb] [--proj ...]` (how LH and the new projects rendered a word); `build_parallels_index.py` now reads all seven projects. The common translator brief: `חוקים-ובריתות/translator-brief--laws.md`.

More tools (2026-10-19, creation corpus projects 3–4):

- **sumerian_workbench.py**: the Sumerian workbench that project 2 asked for. `lemmas <c.xml> FROM-TO` (ETCSL's TEI lemmas per word), `line <c.xml> FROM-TO` (the same, plus for every verb the ORACC morph of the same form wherever it is parsed in epsd2-literary, or "no parsed instance: our analysis"), `morph '<form>'`, `conc '<regex>' [--lemma]` (ETCSL-wide concordance, for formulas), `gloss '<regex>'` (ePSD2 glossary). The ORACC form index is built once into `sources/ORACC/_bulk/epsd2-literary--form-index.json` (gitignored). Run with `python3 -I`.
- **סיפורי-הבריאה-המסופוטמיים/sumerian_tools.py**: `verify KEY [--strict]` (coverage against the ETCSL line list over several annotated files, one `>` per entry, reserved strings licensed by the Sumerian word, name traps, identical transliterations with different Hebrew), `build KEY` (the clean translation generated from `translation--intro.rtl.md` + the annotated `##` headings and `>` lines, with `הערת שוליים` bullets as footnotes), `grep all '<regex>'` (projects 2–4). Add a project as a `PROJECTS` entry. With `translator-brief--sumerian.md` (the reusable brief). `אנומה--1אלישbuild_parallels_index.py` reads projects 3–4 (sigla B&F, EWO).

More tools (2026-10-19, the wisdom corpus, in `ספרות-החכמה-והצדיק-הסובל/`): `wisdom_config.py` (files with eBL line ranges, so one eBL chapter can be split over several annotated files; eBL readers; the Gilgameš corpus's Hebrew for cross-corpus parallels) and `wisdom_tools.py` – `pars KEY` (eBL parallels incl. cross-text `L I.4` links to Gilgameš, and identical normalizations, with the Hebrew already given), `verify [KEY...] [--strict]` (entries, translation = `>` lines, reserved strings licensed by an eBL lemma *or a normalization token*, vocalization traps gated by lemma, repeated lines), `build` (per-file and per-project clean translations generated from `translation--intro-<KEY>.rtl.md` + the annotated file; never edited by hand), `grep '<regex>' [--gilg]`, `lemma '<regex>'` (how a lemma was rendered so far: the cheapest consistency tool); `build_parallels_index.py` (Job first, then Psalms, Qohelet, Proverbs, Lamentations, Isaiah, Jeremiah, the Torah). `translator-brief.md` there is the reusable brief for Akkadian eBL texts.

More tools (2026-10-19, the city-lament corpus, in `קינות-על-ערים/`): `lament_tools.py` – `verify KEY [--strict]` (entries against the ETCSL line list **with segment labels** `A1…` for fragmentary segments, the backticked transliteration equal to ETCSL's line, reserved and gated strings licensed by a Sumerian word, name traps, **fixed refrains checked by regex (FORMULAS)**, translation = `>` lines, identical lines with different Hebrew), `build KEY`, `grep KEY|all` (this corpus + the creation corpus's Sumerian texts), `lines KEY FROM-TO` (ETCSL's lines for a brief); `build_parallels_index.py` (Lamentations first, then Psalms, Jeremiah, Ezekiel, Isaiah, the other Prophets, the Torah, Daniel); `translator-brief--laments.md` (the reusable brief for Sumerian liturgical texts). Add a project as a `PROJECTS` entry.

More tools (2026-10-20, the temple-building corpus, in `בניין-המקדש/`): `temple_tools.py` – `conc GU [<out>]` (the label concordance: ETCSL composite numbers ↔ RIME column-line labels ↔ ORACC, with a per-line similarity check), `data GU FROM TO <out>` (a translator's line-data file:

ETCSL transliteration, ORACC lemma + morph, and the two rank-3 translations – ETCSL's paragraphs mapped from composite numbers to labels, ETCSRI's units – at their first line), `verify GU [--strict]` (as `lament_tools.py`, for labels `A.ix.10`; the reserved Tabernacle words and the gated building words), `build GU`, `grep GU|all`, `lines`; `build_parallels_index.py` (Exodus first, then Kings, Chronicles, Ezekiel, Genesis …); `translator-brief--temple.md` (the reusable brief for Sumerian royal and building texts). ETCSRI per-text HTML + `oracc_html2tr.py` gives the ETCSRI translation of any royal inscription.

More tools (2026-10-21, the netherworld corpus, in `העולם-התחתון/`): `netherworld_tools.py` – `verify IS|IN [--strict]` (entries vs. the eBL labels incl. `NA1`… and the ETCSL labels; the backticked source = the base text; reserved and gated Hebrew licensed by the eBL normalization/lemma or the Sumerian word; name traps; fixed formulas, also across the two languages; translation = `>` lines; identical sources with different Hebrew), `build IS|IN`, `lines IS|IN FROM TO` (base lines with eBL lemmas / ORACC lemma + morph), `grep all '<regex>'` (the `>` lines of every corpus, niqqud stripped); `build_parallels_index.py` – **generates** the per-project parallels tables from the annotated **הקבלה** bullets and merges them by Bible verse into `parallels-index.md` (the key findings kept): no hand-copied tables, so the index cannot drift from the annotations; recommended for new corpora.